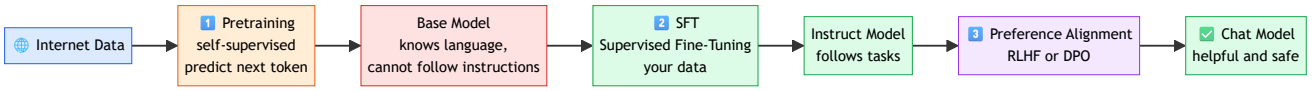
Instruction Fine-Tuning: Easy Visual Guide

> Based on my handwritten notes. Corrections: Transformer paper is **2017** (not 2018). "Llama 4.2" is probably **Llama 3.2**, so check the model name.

---

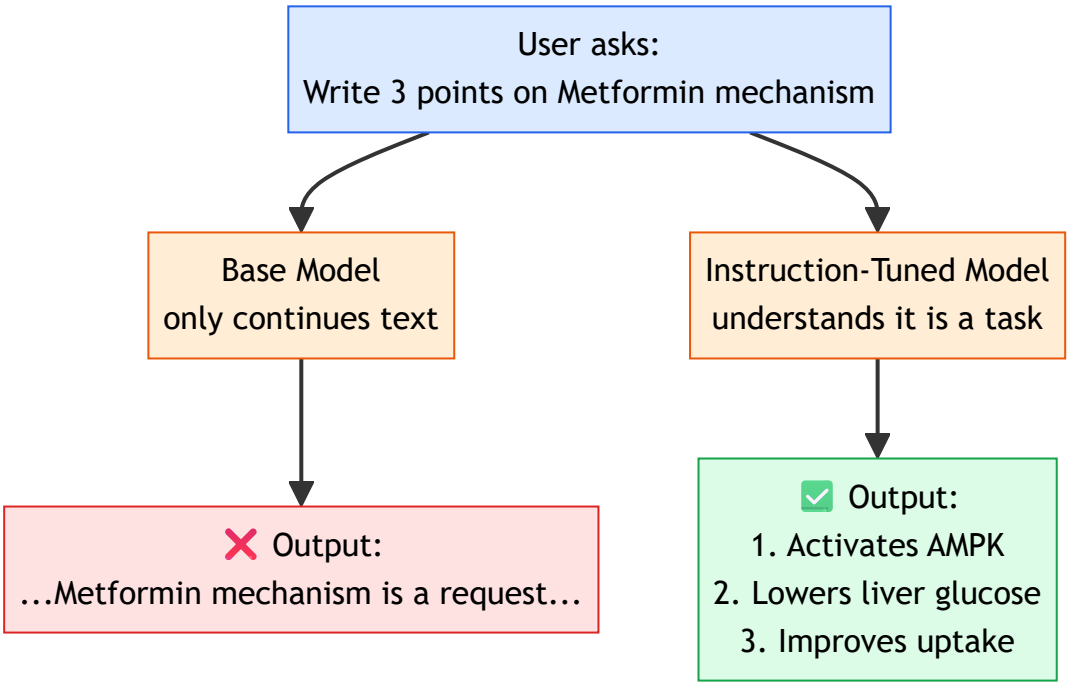
1. The 3 Stages of LLM Training



Read it: Pretraining gives general language. SFT gives your behavior or domain. Alignment gives polish and safety.

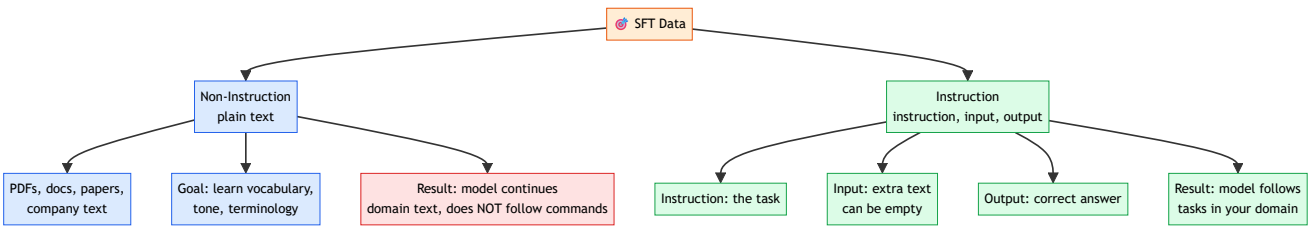
---

2. Why a Base Model Fails (Why We Need Instruction Tuning)



---

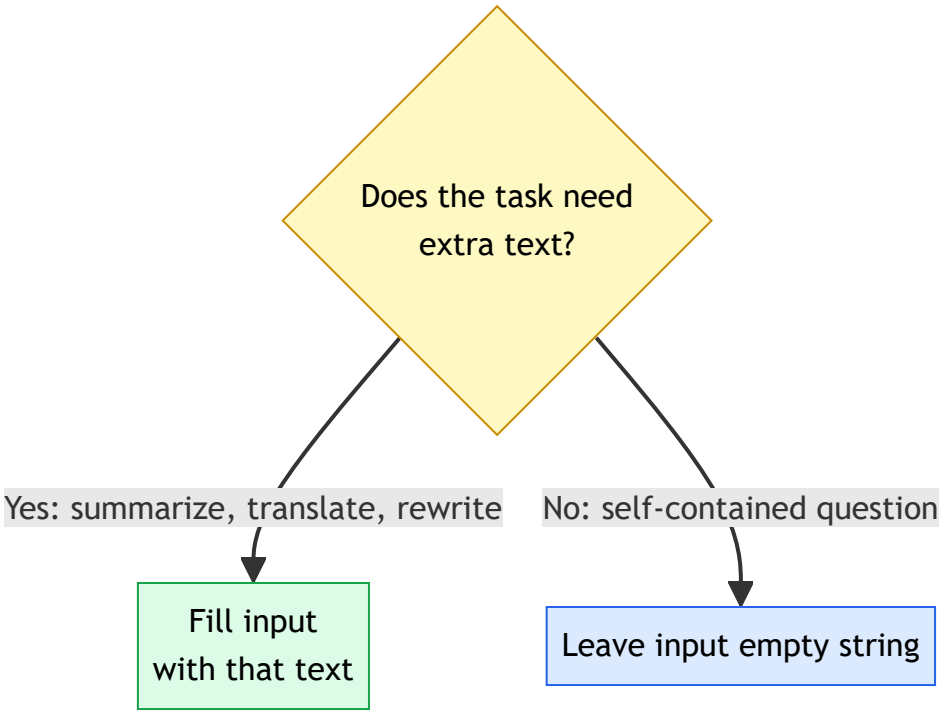
3. Two Types of SFT Data



Non-instruction example (next-token pairs)

Input (context)	Output (next token)
Metformin is one of the most widely prescribed	**oral**
...prescribed oral	**antihyperglycemic**
...prescribed oral antihyperglycemic	**agents**

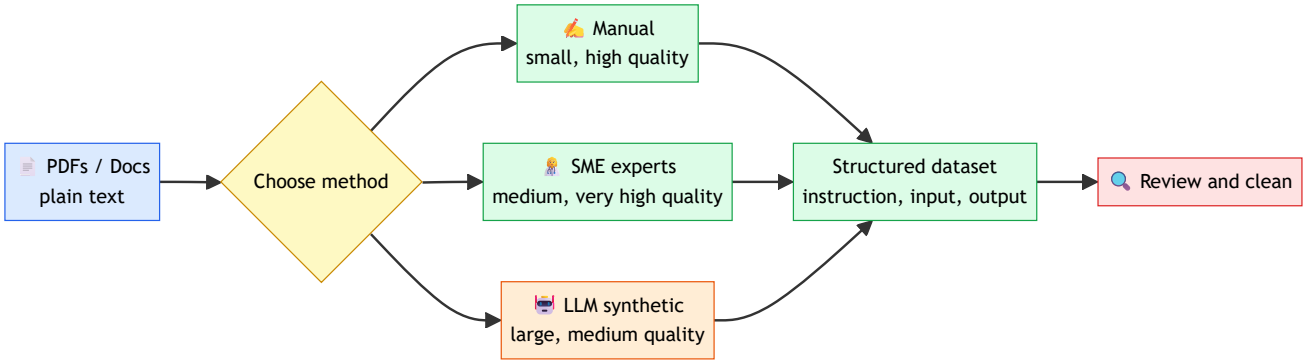
4. When to Use "input"




```
{"instruction": "Summarize the following paragraph.",
 "input": "Metformin activates AMPK, which enhances glucose uptake...",
 "output": "Metformin activates AMPK to improve glucose control."}

{"instruction": "Explain the mechanism of Metformin.",
 "input": "",
 "output": "Metformin activates AMPK and enhances glucose uptake."}
```

5. How to Build the Dataset

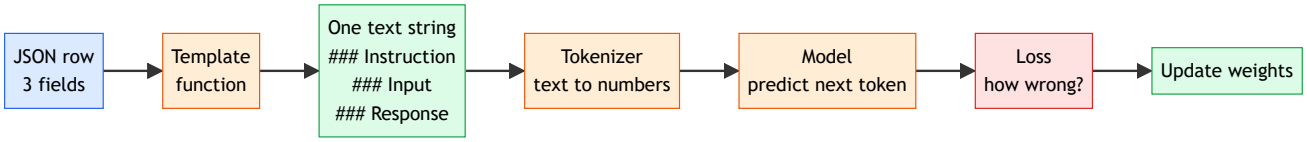


Tip: Always review synthetic data. A few thousand clean examples beat many noisy ones.

---

6. The Key Step: Flatten Into One Text

The model sees one continuous text, not columns.




```
### Instruction:
Summarize the following paragraph.

### Input:
Metformin activates AMPK...

### Response:
Metformin enhances glucose uptake... <EOS>
```

Important: Use the same template at training and inference. Add an **EOS token** so the model learns to stop.

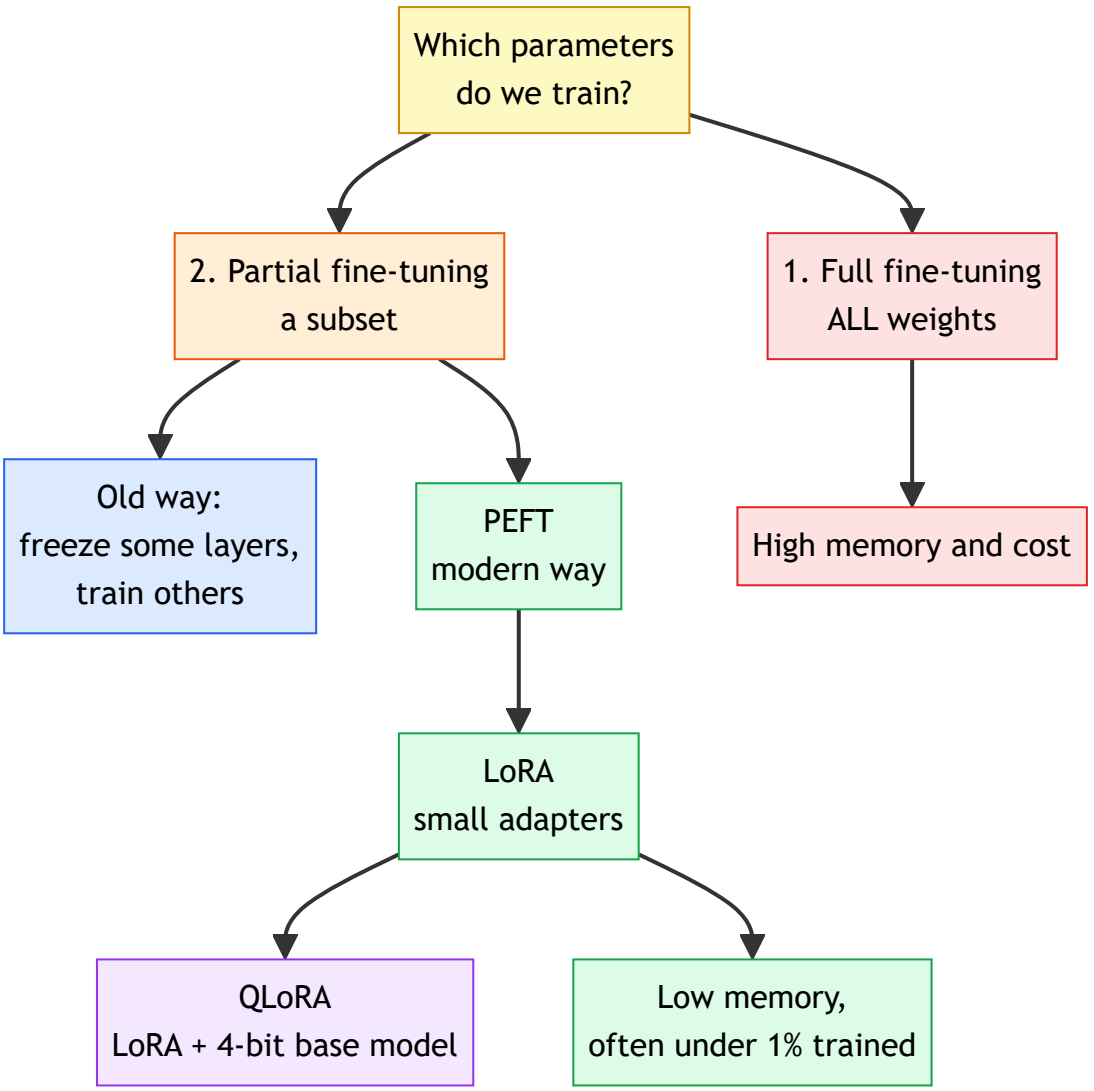
---

7. Prompt Template Formats

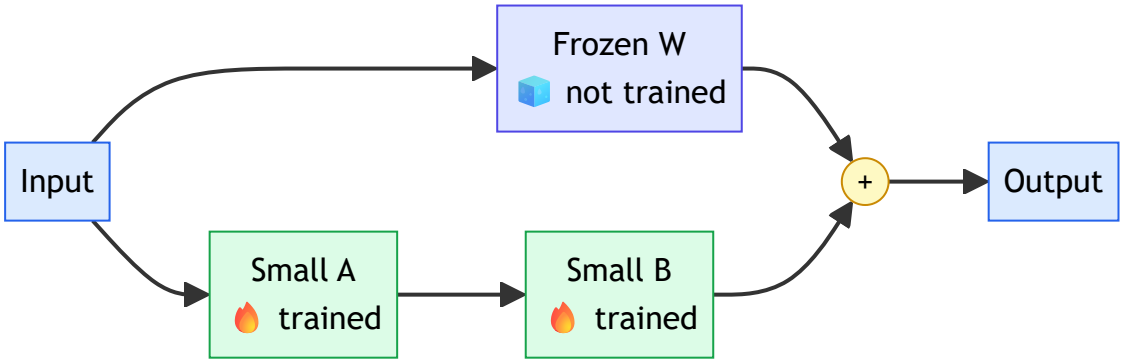
Format	Style
`### Instruction / ### Input / ### Response`	Alpaca
`Instruction / Context / Answer`	Dolly-style
`USER: / ASSISTANT:`	Vicuna-style chat
`System / User / Assistant`	Modern chat models, use `tokenizer.apply_chat_template`

---

8. Full vs Partial vs PEFT (LoRA)



LoRA idea: Freeze the big weight matrix **W**. Train two small matrices **A** and **B**. The new weight is **W + B·A**.



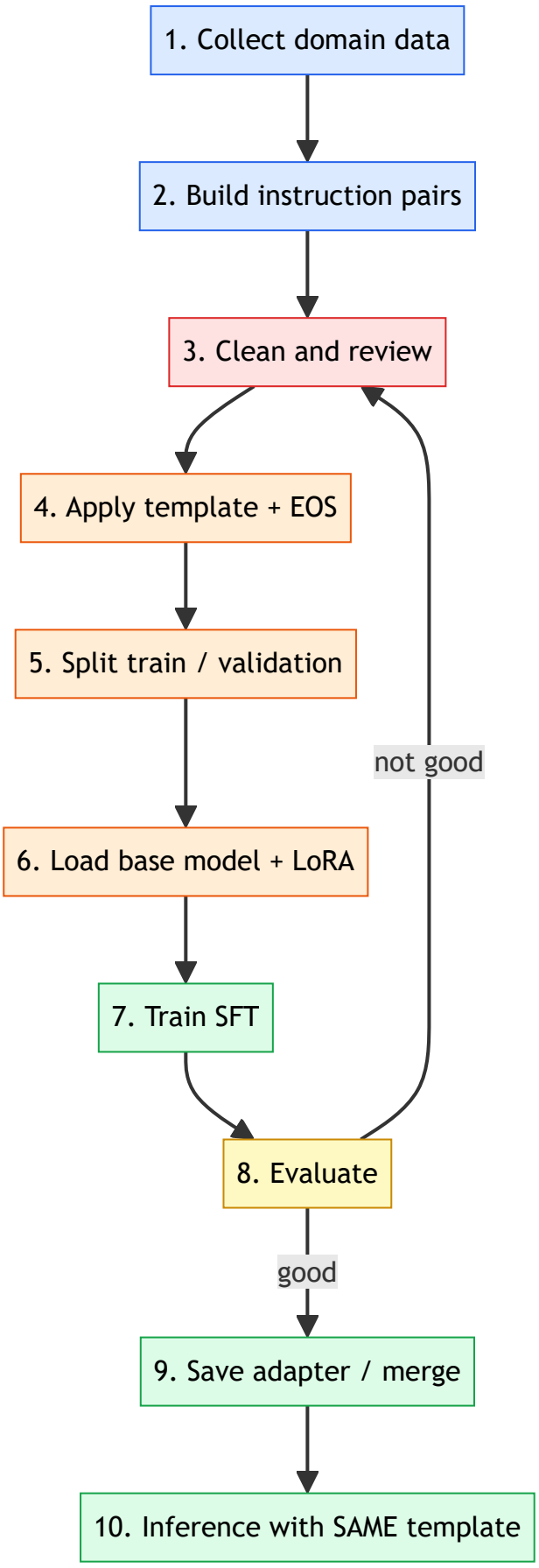
LoRA parameters (typical values, verify for your model)

Parameter	Meaning	Typical
`r`	Adapter size (capacity)	8 to 64
`lora_alpha`	Strength of the update	1 to 2 times `r`
`lora_dropout`	Regularization	0 to 0.1

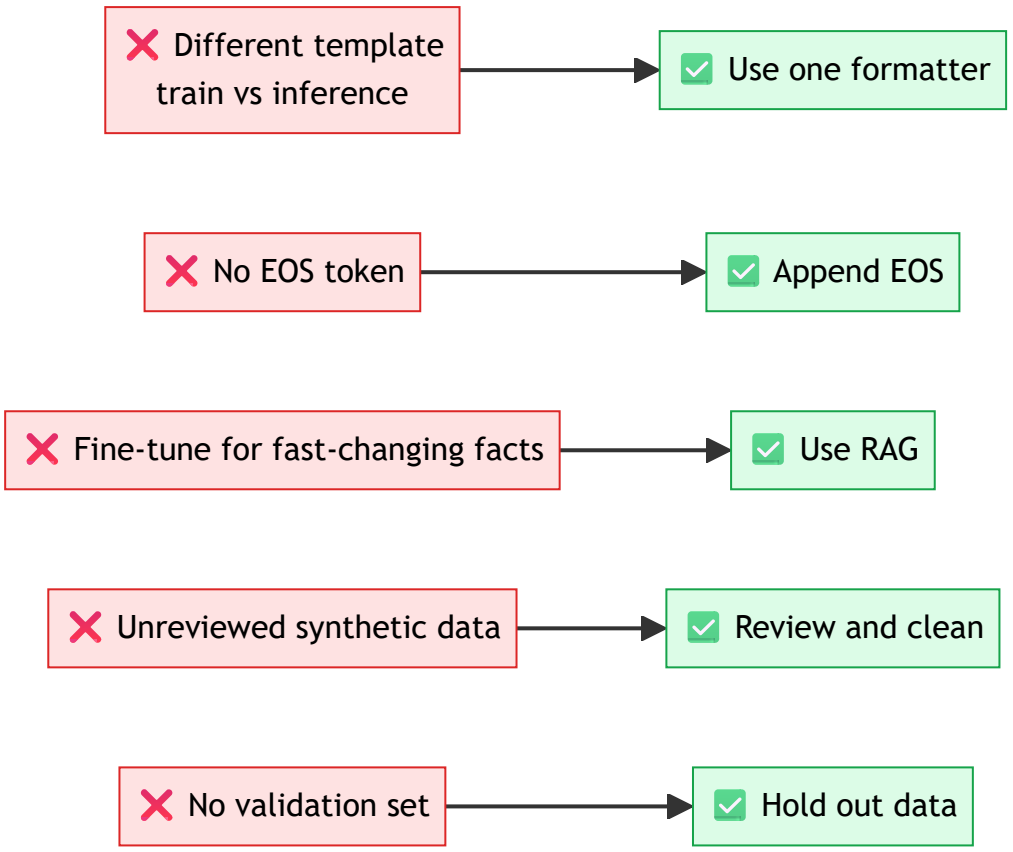
Parameter	Meaning	Typical
`target_modules`	Layers that get adapters	`q_proj`, `v_proj` or more
learning rate	Update step	about 1e-4 to 2e-4
epochs	Passes over data	1 to 3

---

9. Full Project Pipeline


1. Collect domain data
2. Build instruction pairs
3. Clean and review
4. Apply template + EOS
5. Split train / validation
6. Load base model + LoRA
7. Train SFT
8. Evaluate
not good
good
9. Save adapter / merge
10. Inference with SAME template

10. Common Mistakes and Fixes



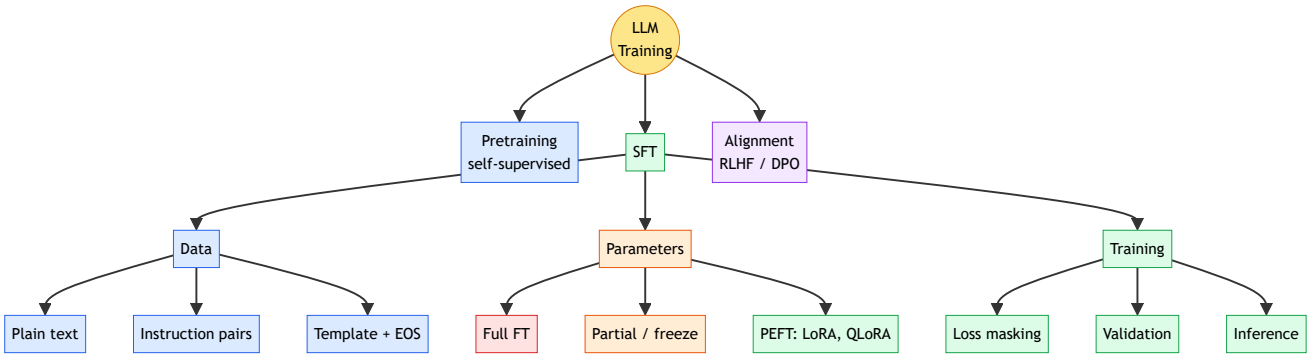
11. Troubleshooting

Problem	Likely cause	Fix
Model repeats the prompt	Template mismatch	Same template at train and inference
Never stops generating	Missing EOS	Add EOS to training text
CUDA out of memory	Full FT, big batch	LoRA/QLoRA, smaller batch, gradient accumulation
Generic or wrong answers	Poor data	Clean and curate
Forgets general skills	High LR or too many epochs	Lower LR, fewer epochs, mix general data

12. Fine-Tuning vs RAG vs Prompting

	Best for	Changes model?
Prompting	Quick behavior control	No
RAG	Fresh or private facts	No
Fine-tuning	Style, format, domain behavior	Yes

13. Knowledge Map



---

14. Quick Revision

1. Base model = only next-token prediction.
2. Instruction tuning = teach the model that a prompt is a task.
3. "Supervised" = there is a human-written target answer.
4. Input empty = self-contained instruction.
5. Flatten each row into one text with a template and EOS.
6. Same template at training and inference.
7. LoRA = freeze W, train small A and B.
8. Data quality matters more than quantity.
9. Facts that change often belong in RAG.
10. Always evaluate on held-out data.

Next topics: chat templates, loss masking, evaluation, DPO.